prominent Academy

If you haven't practiced these 90+ Spring Boot interview questions... rejection is almost guaranteed.

www.prominentacademy.in

+91 98604 38743

## Section 1: Spring Boot Core Concepts (Q1–Q12)

### Q1. What is Spring Boot and how is it different from the Spring Framework?

Spring Boot is an opinionated, convention-over-configuration extension of the Spring Framework that simplifies bootstrapping and development of Spring applications. The Spring Framework provides the core IoC container, AOP, data access, and MVC modules, but requires significant XML or Java configuration. Spring Boot eliminates boilerplate configuration via auto-configuration, embedded servers (Tomcat/Jetty), and Starter POMs so developers can run a production-ready app with minimal setup. Key differences: Spring Boot provides an embedded HTTP server, auto-configuration, Actuator for monitoring, and an opinionated defaults approach, whereas the Spring Framework requires explicit setup.

***Tip: Always mention 'Convention over Configuration' and 'Opinionated Defaults' — interviewers love those keywords.***

### Q2. What does @SpringBootApplication do internally?

@SpringBootApplication is a convenience annotation that combines three annotations: (1) @Configuration — marks the class as a source of bean definitions. (2) @EnableAutoConfiguration — tells Spring Boot to start adding beans based on classpath settings, other beans, and various property settings. (3) @ComponentScan — enables component scanning so Spring can find and register controllers, services, and repositories in the same package and sub-packages.

```
@SpringBootApplication public class MyApp { public static void main(String[] args) { SpringApplication.run(MyApp.class, args); } }
```

***Tip: Mention that excludeAutoConfiguration attribute can be used to exclude specific auto-configurations.***

### Q3. Explain the Spring Boot startup process step by step.

1. SpringApplication.run() is invoked — creates a SpringApplication instance. 2. Determines application type: SERVLET, REACTIVE, or NONE. 3. Loads ApplicationContext initializers and listeners from spring.factories. 4. Publishes ApplicationStartingEvent. 5. Prepares the Environment (loads application.properties/yml, profiles). 6. Prints Banner. 7. Creates ApplicationContext (AnnotationConfigServletWebServerApplicationContext for web apps). 8. Executes auto-configuration. 9. Refreshes context — instantiates all beans. 10. Starts embedded server (Tomcat). 11. Publishes ApplicationStartedEvent and ApplicationReadyEvent.

### Q4. What is the difference between @Component, @Service, @Repository, and @Controller?



All four are specializations of @Component and serve as stereotype annotations, but they convey different semantic meaning and enable specific behavior: @Component — generic Spring-managed component. @Service — marks business logic layer; no extra behavior but documents intent. @Repository — marks data access layer; enables automatic persistence exception translation (Spring wraps JPA/JDBC exceptions into Spring's DataAccessException hierarchy). @Controller — marks MVC web controllers; works with @RequestMapping to handle HTTP requests. @RestController = @Controller + @ResponseBody (returns JSON/XML directly).

***Tip: @Repository's exception translation is the most important difference to mention in interviews.***

## Q5. What is Spring IoC and Dependency Injection? What types of DI does Spring support?

IoC (Inversion of Control) means the Spring container manages object creation and wiring instead of the developer. Dependency Injection is the mechanism: Spring injects required dependencies into a class. Types of DI in Spring: (1) Constructor Injection — dependencies passed via constructor; recommended for mandatory dependencies and immutability. (2) Setter Injection — dependencies set via setter methods; suitable for optional dependencies. (3) Field Injection (@Autowired on fields) — convenient but makes testing harder and hides dependencies.

```
// Constructor Injection (Preferred) @Service public class OrderService { private final PaymentService paymentService; public OrderService(PaymentService paymentService) { this.paymentService = paymentService; } }
```

■ ***Tip: Always recommend Constructor Injection for production code — it allows final fields and easier unit testing.***

## Q6. What is a Spring Bean? What are the different Bean scopes?

A Spring Bean is any object managed by the Spring IoC container. Beans are defined using @Component, @Bean, or XML configuration. Bean Scopes: (1) Singleton (default) — one instance per Spring container. (2) Prototype — new instance every time the bean is requested. (3) Request — one instance per HTTP request (web apps). (4) Session — one instance per HTTP session (web apps). (5) Application — one instance per ServletContext. (6) WebSocket — one instance per WebSocket session.

■■ **Note:** Injecting a prototype bean into a singleton bean does NOT work as expected without using ApplicationContext.getBean() or @Lookup.

## Q7. What is the Bean lifecycle in Spring Boot?



Spring Bean lifecycle: 1. Instantiation — Spring creates the bean instance. 2. Populate Properties — Spring injects dependencies. 3. BeanNameAware.setBeanName() — if implemented. 4. BeanFactoryAware / ApplicationContextAware callbacks. 5. BeanPostProcessor.postProcessBeforeInitialization(). 6. @PostConstruct / InitializingBean.afterPropertiesSet() / init-method. 7. BeanPostProcessor.postProcessAfterInitialization(). 8. Bean ready for use. 9. @PreDestroy / DisposableBean.destroy() / destroy-method on container shutdown.

```
@Component public class MyBean { @PostConstruct public void init() { System.out.println("Bean initialized"); } @PreDestroy public void cleanup() { System.out.println("Bean destroyed"); } }
```

## Q8. What is Spring AOP? Explain key AOP terminologies.

AOP (Aspect-Oriented Programming) allows separating cross-cutting concerns (logging, security, transactions) from business logic. Key terms: Aspect — module encapsulating cross-cutting concern (e.g., LoggingAspect). Join Point — execution point where advice can be applied (e.g., method execution). Pointcut — expression that matches join points. Advice — action taken at a join point: @Before, @After, @AfterReturning, @AfterThrowing, @Around. Weaving — linking aspects with application code (Spring uses runtime proxy-based weaving).

```
@Aspect @Component public class LoggingAspect { @Around("execution(* com.app.service.*.*(..))") public Object log(ProceedingJoinPoint pjp) throws Throwable { long start = System.currentTimeMillis(); Object result = pjp.proceed(); System.out.println("Time: " + (System.currentTimeMillis()-start) + "ms"); return
```

```
result; } }
```

### Q9. What are Profiles in Spring Boot and how do you use them?

Spring Profiles allow grouping bean definitions and configurations for different environments (dev, test, prod). Activate with: spring.profiles.active=prod in application.properties, or -Dspring.profiles.active=prod as JVM arg, or SPRING_PROFILES_ACTIVE env variable. Use @Profile("dev") on @Bean or @Component to conditionally register beans. Use separate property files: application-dev.properties, application-prod.properties.

```
@Configuration @Profile("prod") public class ProdConfig { @Bean public DataSource dataSource() { /* production DB */ } }
```

***Tip: You can activate multiple profiles: spring.profiles.active=dev,swagger***

### Q10. What is application.properties vs application.yml? Which is preferred?

Both serve the same purpose — externalizing configuration. application.properties uses key=value format and is simpler for flat structures. application.yml uses YAML format, supports hierarchical structure and is more readable for nested configs. YAML also supports multi-document files using --- separator to define profiles in a single file. There is no strict preference; yml is commonly used in modern Spring Boot projects for its cleaner structure.

```
# application.yml example spring: datasource: url: jdbc:mysql://localhost:3306/mydb username: root jpa: hibernate: ddl-auto: update
```

### Q11. What is @ConfigurationProperties and how is it different from @Value?

@Value injects a single property value: @Value("${app.name}"). Simple but verbose for multiple properties. @ConfigurationProperties maps a group of related properties to a POJO — cleaner and type-safe. Supports relaxed binding (camelCase, kebab-case, SCREAMING_SNAKE_CASE all map to the same field). Must be enabled with @EnableConfigurationProperties or @ConfigurationPropertiesScan.

```
@ConfigurationProperties(prefix = "app") @Component public class AppConfig { private String name; private int timeout; // getters & setters } # application.properties app.name=MyApp app.timeout=30
```

***Tip: @ConfigurationProperties supports JSR-303 validation with @Validated annotation.***

### Q12. What is the difference between @Bean and @Component?

@Component is a class-level annotation; Spring auto-detects and registers the class as a bean via component scanning. You own the class and can annotate it directly. @Bean is a method-level annotation used inside @Configuration classes. It's used when: you don't own the class (e.g., third-party library), you need conditional bean creation, or you need full control over instantiation logic. Both result in Spring-managed beans, but @Bean provides more flexibility.

```
@Configuration public class AppConfig { @Bean public ObjectMapper objectMapper() { // third-party class return new ObjectMapper(); } }
```

prominent Academy

# Prominent Academy me:

Real production scenarios
Pressure-based mock interviews
Decision-making framework
Interview Prep
Interview calls ( Unlimited Till placement)

**Pay After Placement option**

WhatsApp

**+91 93594 45862**

## ■Section 2: Auto-Configuration & Starter POMs (Q13–Q20)

### Q13. How does Spring Boot Auto-Configuration work internally?

Spring Boot auto-configuration reads the META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports file (spring.factories in older versions) and conditionally registers beans based on @Conditional annotations. Example: DataSourceAutoConfiguration is applied only if a DataSource class is on the classpath AND no DataSource bean is already defined. Key @Conditional annotations: @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty, @ConditionalOnWebApplication.

```
@Configuration @ConditionalOnClass(DataSource.class)
@ConditionalOnMissingBean(DataSource.class) public class DataSourceAutoConfiguration {
... }
```

■ ***Tip: Use spring.autoconfigure.exclude or @SpringBootApplication(exclude=...) to disable specific auto-configurations.***



### Q14. What are Spring Boot Starter POMs? Name some commonly used starters.

Starter POMs are curated dependency descriptors that bundle commonly used libraries together, eliminating version conflicts. Common starters: spring-boot-starter-web (Spring MVC + Tomcat), spring-boot-starter-data-jpa (JPA + Hibernate), spring-boot-starter-security (Spring Security), spring-boot-starter-test (JUnit5 + Mockito + AssertJ), spring-boot-starter-actuator (monitoring), spring-boot-starter-cache, spring-boot-starter-amqp (RabbitMQ), spring-boot-starter-data-redis.

■ ***Tip: Starters follow naming convention: spring-boot-starter-* for official, *-spring-boot-starter for third-party.***

### Q15. How do you create a custom Spring Boot Auto-Configuration?

1. Create your @Configuration class with @Conditional annotations. 2. Create META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports file. 3. Add your configuration class fully qualified name in that file. 4. Package it as a jar. This is how third-party libraries integrate with Spring Boot (e.g., Flyway, Liquibase, Zipkin auto-configure themselves).

```
// Step 1: Config class @Configuration @ConditionalOnClass(MyService.class)
@ConditionalOnMissingBean(MyService.class) public class MyAutoConfig { @Bean public
MyService myService() { return new MyService(); } } // Step 2: File:
META-INF/spring/...AutoConfiguration.imports com.example.MyAutoConfig
```

### Q16. What is @ConditionalOnProperty? Give a real use case.

@ConditionalOnProperty registers a bean only if a specific property is set to a given value in application.properties. Real use case: Enable/disable a feature flag or a mock service in non-production environments.

```
@Bean @ConditionalOnProperty(name = "feature.payment.mock", havingValue = "true",
matchIfMissing = false) public PaymentService mockPaymentService() { return new
MockPaymentService(); }
```

### Q17. What is the use of spring.factories? Is it still used in Spring Boot 3.x?

In Spring Boot 2.x, META-INF/spring.factories was used to register auto-configurations, listeners, initializers, and failure analyzers. In Spring Boot 3.x (Spring Framework 6), auto-configuration registration was moved to META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports for better startup performance. spring.factories is still supported but deprecated for auto-configuration entries. Other entries like ApplicationListener are still registered via spring.factories.

### Q18. What is the difference between @Import and @ImportResource?

@Import — imports one or more @Configuration classes, ImportSelector implementations, or ImportBeanDefinitionRegistrar implementations programmatically. @ImportResource — imports Spring XML configuration files into a Java-based configuration. Mainly used during migration from XML-based Spring config to Java config.

```
@Configuration @Import({SecurityConfig.class, CacheConfig.class}) public class AppConfig { } @Configuration @ImportResource("classpath:legacy-beans.xml") public class LegacyConfig { }
```

### Q19. How does Spring Boot handle multiple DataSource configurations?

For multiple datasources: define two DataSource beans, mark one @Primary to avoid ambiguity. Use @Qualifier to inject the correct one. Disable Spring Boot's DataSource auto-configuration to avoid conflicts. Configure separate JPA EntityManagerFactory and TransactionManager for each datasource.

```
@Configuration @EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class}) public class MultiDbConfig { @Bean @Primary @ConfigurationProperties("spring.datasource.primary") public DataSource primaryDs() { return DataSourceBuilder.create().build(); } @Bean @ConfigurationProperties("spring.datasource.secondary") public DataSource secondaryDs() { return DataSourceBuilder.create().build(); } }
```

### Q20. What is Banner in Spring Boot? How do you customize or disable it?

Spring Boot prints an ASCII art banner on startup. To customize: place a banner.txt file in src/main/resources. To disable: set spring.main.banner-mode=off in application.properties, or SpringApplication.setBannerMode(Banner.Mode.OFF) programmatically. ANSI colors are supported in banner.txt using ${AnsiColor.RED} variables.

**Note:** In production deployments, disabling the banner reduces log noise.

## Section 3: Spring Boot REST API Development (Q21–Q32)

### Q21. What is the difference between @RestController and @Controller?

@Controller is a stereotype annotation for MVC controllers that return view names (HTML pages via Thymeleaf/JSP). @RestController = @Controller + @ResponseBody — every method automatically serializes the return value to JSON/XML and writes it to the HTTP response. Use @Controller for web MVC, @RestController for REST APIs.

### Q22. Explain the most commonly used HTTP method annotations in Spring Boot.

@GetMapping — HTTP GET (fetch data). @PostMapping — HTTP POST (create resource). @PutMapping — HTTP PUT (full update of resource). @PatchMapping — HTTP PATCH (partial update). @DeleteMapping — HTTP DELETE (remove resource). All are composed annotations of @RequestMapping(method=RequestMethod.*).

```
@RestController @RequestMapping("/api/users") public class UserController {
@GetMapping("/{id}") public User getUser(@PathVariable Long id) {} @PostMapping public
User createUser(@RequestBody @Valid UserDto dto) {} @PutMapping("/{id}") public User
updateUser(@PathVariable Long id, @RequestBody UserDto dto) {} @DeleteMapping("/{id}")
public void deleteUser(@PathVariable Long id) {} }
```

### Q23. What is the difference between @PathVariable, @RequestParam, and @RequestBody?



@PathVariable — extracts values from URI template variables. E.g., /users/{id}. @RequestParam — extracts query parameters from URL. E.g., /users?page=0&size;=10. @RequestBody — deserializes the HTTP request body (typically JSON) into a Java object. Combined example: GET /products/42?currency=INR — @PathVariable captures 42, @RequestParam captures INR.

### Q24. What is ResponseEntity and when should you use it?

ResponseEntity represents the entire HTTP response: status code, headers, and body. Use it when you need fine-grained control over the HTTP response — setting custom status codes, headers, or conditionally returning different response bodies.

```
@GetMapping("/{id}") public ResponseEntity getUser(@PathVariable Long id) { return
userService.findById(id) .map(user -> ResponseEntity.ok(user))
.orElse(ResponseEntity.notFound().build()); }
```

### Q25. How do you implement global exception handling in Spring Boot?

@ControllerAdvice (or @RestControllerAdvice) combined with @ExceptionHandler methods provides centralized exception handling across all controllers. This eliminates try-catch blocks in controllers and returns consistent error responses.

```
@RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(ResourceNotFoundException.class) public ResponseEntity
handleNotFound(ResourceNotFoundException ex) { return
ResponseEntity.status(HttpStatus.NOT_FOUND) .body(new ErrorResponse(ex.getMessage(),
404)); } @ExceptionHandler(MethodArgumentNotValidException.class) public
ResponseEntity> handleValidation(MethodArgumentNotValidException ex) { Map errors =
new HashMap<>(); ex.getBindingResult().getFieldErrors() .forEach(e ->
```

```
errors.put(e.getField(), e.getDefaultMessage())); return
ResponseEntity.badRequest().body(errors); } }
```

■ *Tip: Use @RestControllerAdvice instead of @ControllerAdvice for REST APIs to avoid needing @ResponseBody on each handler.*

### Q26. How do you implement request validation in Spring Boot?

Use Bean Validation (JSR-380) with Hibernate Validator. Add @Valid or @Validated on @RequestBody in controller. Use constraint annotations on DTO fields: @NotNull, @NotBlank, @Size, @Min, @Max, @Email, @Pattern.

```
public class UserDto { @NotBlank(message = "Name is required") @Size(min=2, max=50)
private String name; @Email(message = "Invalid email") private String email; @Min(18)
@Max(120) private int age; } // Controller @PostMapping public User create(@RequestBody
@Valid UserDto dto) { ... }
```

### Q27. What is HATEOAS? How do you implement it in Spring Boot?

HATEOAS (Hypermedia as the Engine of Application State) is a REST constraint where responses include links to related actions/resources, making the API self-discoverable. Spring HATEOAS library provides EntityModel, CollectionModel, and Link classes. Add spring-boot-starter-hateoas dependency.

```
@GetMapping("/{id}") public EntityModel getUser(@PathVariable Long id) { User user =
userService.findById(id); return EntityModel.of(user,
linkTo(methodOn(UserController.class).getUser(id)).withSelfRel(),
linkTo(methodOn(UserController.class).getAllUsers()).withRel("all-users") ); }
```

### Q28. How do you implement pagination and sorting in Spring Boot REST APIs?

Use Spring Data's Pageable and Page interfaces. Controller accepts Pageable automatically with @PageableDefault. Request: GET /products?page=0&size;=10&sort;=price,asc

```
@GetMapping public Page getProducts(@PageableDefault(size=10, sort="name") Pageable
pageable) { return productRepository.findAll(pageable); } // Repository public
interface ProductRepo extends JpaRepository { Page findByCategory(String category,
Pageable pageable); }
```

### Q29. What is content negotiation in Spring Boot?



Content negotiation allows the same endpoint to produce different representations (JSON, XML) based on the client's Accept header. Spring MVC handles this automatically. Add jackson-dataformat-xml to support XML. Client sets: Accept: application/xml or Accept: application/json.

```
# pom.xml com.fasterxml.jackson.dataformat jackson-dataformat-xml # Controller returns
User - Spring negotiates based on Accept header
```

### Q30. How do you implement API versioning in Spring Boot?

Common strategies: (1) URI versioning — /api/v1/users, /api/v2/users. Most common and visible. (2) Request Parameter — /api/users?version=1. (3) Header versioning — Custom header: API-Version: 1. (4) Content-Type versioning — Accept: application/vnd.company.v1+json. URI versioning is simplest to implement and debug.

```
@RestController @RequestMapping("/api/v1/users") public class UserV1Controller { ... }
@RestController @RequestMapping("/api/v2/users") public class UserV2Controller { ... }
```

### Q31. What is CORS and how do you configure it in Spring Boot?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP requests to different origins. Spring Boot provides multiple ways to configure CORS: @CrossOrigin on controller/method level, or global configuration via WebMvcConfigurer.

```
// Global CORS config @Configuration public class CorsConfig implements
WebMvcConfigurer { @Override public void addCorsMappings(CorsRegistry registry) {
registry.addMapping("/api/**") .allowedOrigins("https://myapp.com")
.allowedMethods("GET","POST","PUT","DELETE") .allowCredentials(true) .maxAge(3600); }
}
```

▪▪ **Note:** Never use allowedOrigins("*") with allowCredentials(true) — it's a security risk.

### Q32. How does Jackson serialize/deserialize objects in Spring Boot? What are useful Jackson annotations?

Spring Boot auto-configures Jackson ObjectMapper for JSON serialization/deserialization. Useful annotations: @JsonProperty (custom field name), @JsonIgnore (exclude field), @JsonIgnoreProperties (ignore unknown fields), @JsonInclude(NON_NULL) (exclude null fields), @JsonFormat (date formatting), @JsonAlias (multiple input names).

```
@JsonInclude(JsonInclude.Include.NON_NULL) public class UserDto {
@JsonProperty("full_name") private String name; @JsonIgnore private String password;
@JsonFormat(pattern="yyyy-MM-dd") private LocalDate dob; }
```

## ■Section 4: Spring Data JPA & Database (Q33–Q43)

### Q33. What is Spring Data JPA? How is it different from Hibernate?

Hibernate is a JPA (Java Persistence API) implementation — it handles ORM (Object-Relational Mapping). Spring Data JPA is an abstraction layer on top of JPA/Hibernate that reduces boilerplate. By extending JpaRepository, you get CRUD operations, pagination, sorting, and custom queries for free without writing a single line of SQL. Hibernate is the underlying engine; Spring Data JPA provides the repository pattern abstraction.

### Q34. What are the different types of repository interfaces in Spring Data?

Repository — marker interface. CrudRepository — basic CRUD: save, findById, findAll, delete. PagingAndSortingRepository — adds pagination and sorting. JpaRepository — extends above + JPA-specific methods (flush, saveAndFlush, deleteInBatch). For most use cases, extending JpaRepository is recommended.

```
public interface ProductRepository extends JpaRepository { List findByCategory(String category); Optional findBySkuIgnoreCase(String sku); Page findByPriceBetween(Double min, Double max, Pageable p); @Query("SELECT p FROM Product p WHERE p.stock = 0") List findOutOfStock(); }
```

### Q35. What is the difference between @Query with JPQL and Native Query?

JPQL (Java Persistence Query Language) operates on entity objects and field names, not table/column names — it's database-agnostic. Native Query uses actual SQL — table names and column names — and is database-specific. Use native queries for performance-critical queries, database-specific functions, or complex joins not expressible in JPQL.

```
// JPQL @Query("SELECT u FROM User u WHERE u.email = :email") Optional findByEmail(@Param("email") String email); // Native Query @Query(value = "SELECT * FROM users WHERE email = :email", nativeQuery = true) Optional findByEmailNative(@Param("email") String email);
```

### Q36. What are JPA entity relationships? Explain with examples.

@OneToOne — one entity related to exactly one other. E.g., User ↔ UserProfile. @OneToMany / @ManyToOne — e.g., one Department has many Employees; each Employee belongs to one Department. @ManyToMany — e.g., Student ↔ Course. Requires a join table.

```
@Entity public class Department { @Id @GeneratedValue private Long id; private String name; @OneToMany(mappedBy="department", cascade=CascadeType.ALL, fetch=FetchType.LAZY) private List employees; } @Entity public class Employee { @ManyToOne(fetch=FetchType.LAZY) @JoinColumn(name="dept_id") private Department department; }
```

■ ***Tip: Always use FetchType.LAZY for collections to avoid N+1 queries and performance issues.***

### Q37. What is the N+1 Select problem and how do you solve it?

N+1 problem: Fetching N parent entities triggers N additional queries to fetch each parent's children. Example: Fetching 100 orders and their order items results in 100 + 1 = 101 queries. Solutions: (1) JPQL JOIN FETCH — fetches in a single query. (2) @EntityGraph — specify associations to eager-load per query. (3) @BatchSize — batch loads children. (4) Hibernate @Fetch(FetchMode.SUBSELECT).

```
// Solution 1: JOIN FETCH @Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.status = :status") List findWithItems(@Param("status") String status); // Solution 2: EntityGraph @EntityGraph(attributePaths = {"items", "items.product"}) @Query("SELECT o FROM Order o") List findAllWithItemsAndProducts();
```

▪▪ **Note:** The N+1 problem is one of the most common JPA performance issues. Interviewers love this question.

### Q38. What is the difference between EntityManager.persist(), merge(), and save() in Spring Data?



persist() — makes a transient entity managed/persistent. Fails if entity already has an ID (detached). merge() — merges the state of a detached entity into the current persistence context; returns a new managed instance. Spring Data's save() — internally calls persist() if entity is new (no ID), merge() if entity already has an ID. save() returns the managed entity — always use the returned object.

### Q39. What is @Transactional? What are its propagation levels?

@Transactional manages database transactions declaratively. Propagation levels: REQUIRED (default — join existing or create new), REQUIRES_NEW (always new transaction, suspends existing), SUPPORTS (use existing if present, else non-transactional), NOT_SUPPORTED (suspend existing), MANDATORY (must have existing), NEVER (must not have existing), NESTED (savepoint within existing). rollbackFor — specify exceptions that trigger rollback. Default: RuntimeException and Error.

```
@Service public class OrderService { @Transactional(propagation = Propagation.REQUIRED, rollbackFor = Exception.class, readOnly = false) public Order createOrder(OrderDto dto) { ... } @Transactional(readOnly = true) // optimization for reads public Order getOrder(Long id) { ... } }
```

▪▪ **Note:** @Transactional only works on public methods when applied via Spring AOP proxy. Self-invocation bypasses the proxy.

### Q40. What is the difference between FetchType.LAZY and FetchType.EAGER?

EAGER — associated entities are loaded immediately when the parent is loaded, regardless of whether you need them. LAZY — associated entities are loaded only when accessed, using a proxy. This is the recommended default for collections (@OneToMany, @ManyToMany). @ManyToOne and @OneToOne default to EAGER — always override to LAZY for performance.

■ ***Tip: LazyInitializationException occurs when you access a LAZY association outside a transaction (after session closes). Solve with @Transactional or DTO projection.***

### Q41. What are Spring Data Projections? Explain interface-based and DTO projections.

Projections fetch only required columns instead of the entire entity — improves performance. Interface-based: define an interface with getter methods matching entity fields; Spring Data implements it dynamically. DTO/Class-based: define a class with a constructor matching the fields; use @Query with constructor expression.

```
// Interface Projection public interface UserSummary { String getName(); String getEmail(); } List findByRole(String role); // DTO Projection public record UserDto(String name, String email) {} @Query("SELECT new com.app.dto.UserDto(u.name, u.email) FROM User u WHERE u.role = :role") List findUserSummary(@Param("role") String role);
```

### Q42. What is Flyway / Liquibase? How does Spring Boot integrate with them?

Flyway and Liquibase are database migration tools that manage schema evolution with versioned scripts. Spring Boot auto-configures Flyway when spring-boot-starter-data-jpa and flyway-core are on the classpath. Flyway: SQL migration scripts in classpath:db/migration/V1__init.sql, V2__add_column.sql format. Migrations run automatically on startup before the application starts serving requests.

```
-- db/migration/V1__create_users.sql CREATE TABLE users ( id BIGINT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

***Tip: Never modify existing migration scripts — always create new versioned scripts for changes.***

### Q43. What is Optimistic vs Pessimistic locking in JPA?

Optimistic Locking — assumes conflicts are rare. Uses a @Version field to detect concurrent modifications at commit time. If version mismatch, throws OptimisticLockException. No DB lock acquired. Pessimistic Locking — acquires a database lock on read to prevent concurrent access. Uses LockModeType.PESSIMISTIC_WRITE or PESSIMISTIC_READ. Optimistic locking is preferred for read-heavy workloads; pessimistic for write-heavy, critical sections.

```
@Entity public class Product { @Version private Long version; // Optimistic locking // ... other fields } // Pessimistic locking in repository
@Lock(LockModeType.PESSIMISTIC_WRITE) @Query("SELECT p FROM Product p WHERE p.id = :id") Optional findByIdForUpdate(@Param("id") Long id);
```

## Section 5: Spring Security (Q44–Q52)

### Q44. What is Spring Security? Explain how it works at a high level.

Spring Security is a powerful authentication and authorization framework for Spring applications. It works via a chain of Servlet Filters (SecurityFilterChain) that intercept every HTTP request. Key filters: UsernamePasswordAuthenticationFilter, BasicAuthenticationFilter, JwtAuthenticationFilter (custom). Flow: Request → FilterChain → AuthenticationManager → AuthenticationProvider → UserDetailsService → SecurityContext.

### Q45. What is the difference between Authentication and Authorization in Spring Security?

Authentication — verifying WHO the user is (login: username/password, JWT token, OAuth2 token). Authorization — determining WHAT the authenticated user is allowed to do (roles and permissions). Spring Security handles auth via AuthenticationManager and access control via AccessDecisionManager / method security annotations (@PreAuthorize, @Secured).

### Q46. How do you implement JWT-based authentication in Spring Boot?

1. Add jjwt library dependency. 2. Create JwtUtil to generate and validate tokens. 3. Create JwtAuthenticationFilter extending OncePerRequestFilter. 4. Configure SecurityFilterChain to use stateless session and add the filter. 5. Create /auth/login endpoint that returns JWT on valid credentials.

```
@Component public class JwtFilter extends OncePerRequestFilter { @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) throws ServletException, IOException { String header = req.getHeader("Authorization"); if (header != null && header.startsWith("Bearer ")) { String token = header.substring(7); if (jwtUtil.validateToken(token)) { UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(jwtUtil.getUsername(token), null, authorities); SecurityContextHolder.getContext().setAuthentication(auth); } } chain.doFilter(req, res); } }
```



### Q47. What is UserDetailsService and how do you customize it?

UserDetailsService is a core Spring Security interface with one method: loadUserByUsername(String username). Spring Security calls this during authentication to load user details from your data source (database). Implement it to load User from DB and return a UserDetails object with username, encoded password, and granted authorities.

```
@Service public class CustomUserDetailsService implements UserDetailsService { @Autowired private UserRepository userRepo; @Override public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException { User user = userRepo.findByEmail(username) .orElseThrow(() -> new UsernameNotFoundException("User not found")); return org.springframework.security.core.userdetails.User .withUsername(user.getEmail()) .password(user.getPassword()) .roles(user.getRole()) .build(); } }
```

### Q48. How do you configure method-level security in Spring Boot?

Enable with @EnableMethodSecurity (Spring Boot 3.x) or @EnableGlobalMethodSecurity (older). @PreAuthorize — SpEL expression evaluated BEFORE method execution. @PostAuthorize — evaluated AFTER method returns. @Secured — role-based security. @RolesAllowed — JSR-250 annotation.

```
@EnableMethodSecurity @Configuration public class SecurityConfig {} @Service public class ReportService { @PreAuthorize("hasRole('ADMIN') or hasPermission(#id, 'read')") public Report getReport(Long id) { ... } @PreAuthorize("hasAnyRole('ADMIN','MANAGER')") public void deleteReport(Long id) { ... } }
```

### Q49. What is CSRF? When should you disable it in Spring Boot?

CSRF (Cross-Site Request Forgery) is an attack where a malicious site tricks the user's browser into making requests to your app using their cookies. Spring Security enables CSRF protection by default using a synchronizer token pattern. CSRF protection is important for stateful apps (cookie-based sessions). For stateless REST APIs using JWT (no cookies/sessions), CSRF can be safely disabled.

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http .csrf(csrf -> csrf.disable()) // Safe for JWT-based stateless APIs .sessionManagement(session -> session .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) .authorizeHttpRequests(auth -> auth .requestMatchers("/api/auth/**").permitAll() .anyRequest().authenticated()); return http.build(); }
```



### Q50. What is OAuth2 and how does Spring Boot support it?

OAuth2 is an authorization framework that allows third-party apps to access user resources without exposing credentials. Spring Boot supports OAuth2 via spring-boot-starter-oauth2-client (login with Google/GitHub/Facebook) and spring-boot-starter-oauth2-resource-server (protect APIs with JWT/opaque tokens). Roles: Resource Owner (user), Client (your app), Authorization Server (Google), Resource Server (your API).

### Q51. What is PasswordEncoder? Which implementation should you use?

PasswordEncoder is a Spring Security interface for encoding/verifying passwords. Never store plain-text passwords. BCryptPasswordEncoder — uses BCrypt hashing with a work factor (default 10); recommended for most applications. Argon2PasswordEncoder — more memory-hard; better for high-security requirements. SCryptPasswordEncoder — memory and CPU hard. Use DelegatingPasswordEncoder (Spring default) to support multiple encodings during migration.

```
@Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(12); // strength factor 12 } // Usage in service user.setPassword(passwordEncoder.encode(rawPassword)); // Verification boolean matches = passwordEncoder.matches(rawPassword, storedHash);
```

### Q52. What is the SecurityContextHolder and how does it work?

SecurityContextHolder stores the security context (authentication details) for the current thread using a ThreadLocal by default. This means each thread has its own SecurityContext. After successful authentication, the Authentication object (containing principal, credentials, authorities) is stored here. It can be accessed anywhere in the application to get the current user.

```
// Get current authenticated user Authentication auth =
SecurityContextHolder.getContext().getAuthentication(); String username =
auth.getName(); Collection roles = auth.getAuthorities();
```

■■ **Note:** In async methods, the SecurityContext is NOT automatically propagated to new threads. Use SecurityContextHolder.setStrategyName(MODE_INHERITABLETHREADLOCAL) or copy the context manually.

## ■Section 6: Spring Boot Actuator & Monitoring (Q53–Q59)

### Q53. What is Spring Boot Actuator? What endpoints does it expose?

Actuator provides production-ready monitoring and management endpoints over HTTP or JMX. Key endpoints: /actuator/health (app health), /actuator/info (app info), /actuator/metrics (JVM, HTTP, custom metrics), /actuator/env (environment properties), /actuator/beans (all beans), /actuator/mappings (URL mappings), /actuator/loggers (logger levels), /actuator/threaddump, /actuator/heapdump, /actuator/shutdown (disabled by default).

```
# application.properties
management.endpoints.web.exposure.include=health,info,metrics,loggers
management.endpoint.health.show-details=always management.info.env.enabled=true
```

### Q54. How do you create a custom Health Indicator in Spring Boot Actuator?

Implement the HealthIndicator interface and override the health() method. Spring Boot auto-detects and includes it in /actuator/health.

```
@Component public class ExternalServiceHealthIndicator implements HealthIndicator {
@Override public Health health() { try { boolean up = externalService.ping(); return up
? Health.up().withDetail("service", "reachable").build() :
Health.down().withDetail("service", "unreachable").build(); } catch (Exception e) {
return Health.down(e).build(); } } }
```

### Q55. How do you create custom metrics with Micrometer in Spring Boot?

Spring Boot Actuator integrates with Micrometer (a metrics facade). Inject MeterRegistry and create counters, gauges, timers.



```
@Service public class OrderService { private final Counter orderCounter; private final
Timer orderTimer; public OrderService(MeterRegistry registry) { this.orderCounter =
Counter.builder("orders.created") .description("Total orders created") .tag("type",
"online") .register(registry); this.orderTimer =
registry.timer("orders.processing.time"); } public Order createOrder(OrderDto dto) {
return orderTimer.record(() -> { orderCounter.increment(); return processOrder(dto);
}); } }
```

### Q56. How do you integrate Spring Boot Actuator with Prometheus and Grafana?

1. Add micrometer-registry-prometheus dependency. 2. Expose /actuator/prometheus endpoint. 3. Configure Prometheus to scrape this endpoint. 4. Connect Grafana to Prometheus as a data source. 5. Import Spring Boot dashboard (ID: 4701) in Grafana.

```
# pom.xml io.micrometer micrometer-registry-prometheus # application.properties
management.endpoints.web.exposure.include=prometheus,health
management.metrics.export.prometheus.enabled=true
```

### Q57. What is the difference between /health and /info endpoints?

/health — reports application health status (UP/DOWN/OUT_OF_SERVICE). Aggregates health from DB, disk space, external services, and custom indicators. Used by load balancers and orchestrators (Kubernetes liveness/readiness probes). /info — provides arbitrary application information: version, build details, git commit. Populated via application.properties info.* keys or InfoContributor beans. Used for documentation/debugging.

### Q58. How do you use Spring Boot Actuator for Kubernetes liveness and readiness probes?

Spring Boot 2.3+ auto-configures liveness and readiness probes for Kubernetes. Liveness probe (/actuator/health/liveness) — Kubernetes restarts the pod if this fails. Readiness probe (/actuator/health/readiness) — Kubernetes stops sending traffic if this fails. Enable with management.health.probes.enabled=true (auto-enabled in Kubernetes environment).

```
# application.properties management.health.probes.enabled=true
management.endpoint.health.probes.add-additional-paths=true # Kubernetes probe config:
# livenessProbe: # httpGet: {path: /actuator/health/liveness, port: 8080} #
readinessProbe: # httpGet: {path: /actuator/health/readiness, port: 8080}
```

### Q59. What is Spring Boot Admin? How is it different from Actuator?

Spring Boot Admin is an open-source community project that provides a nice web UI on top of Actuator endpoints. Actuator exposes raw JSON data over HTTP. Spring Boot Admin aggregates and visualizes this data from multiple Spring Boot services in a user-friendly dashboard showing health, metrics, logs, and thread dumps. Setup: Create a Spring Boot Admin Server app, register client apps by adding spring-boot-admin-starter-client dependency.

## Section 7: Testing in Spring Boot (Q60–Q68)

### Q60. What testing libraries does Spring Boot provide out of the box?

spring-boot-starter-test includes: JUnit 5 (Jupiter) — test framework, Mockito — mocking framework, AssertJ — fluent assertions, Hamcrest — matchers, Spring Test & Spring Boot Test — integration testing utilities, JSONassert — JSON assertions, JsonPath — JSON path evaluation, Testcontainers (optional) — Docker-based integration tests.

### Q61. What is the difference between @SpringBootTest, @WebMvcTest, @DataJpaTest, and @JsonTest?

@SpringBootTest — loads full application context. Used for integration tests. Slow. @WebMvcTest(Controller.class) — loads only the web layer (controllers, filters, @ControllerAdvice). Fast. Mocks service layer. @DataJpaTest — loads only JPA layer (repositories, entities). Uses in-memory H2 by default. @JsonTest — tests JSON serialization/deserialization only.

```
@WebMvcTest(UserController.class) class UserControllerTest { @Autowired MockMvc mockMvc; @MockBean UserService userService; @Test void shouldReturnUser() throws Exception { when(userService.findById(1L)).thenReturn(new User(1L, "Alice")); mockMvc.perform(get("/api/users/1")) .andExpect(status().isOk()) .andExpect(jsonPath("$.name").value("Alice")); } }
```

### Q62. What is MockMvc and how do you use it?



MockMvc simulates HTTP requests to Spring MVC controllers without starting a real HTTP server. It tests the full web layer (controllers, filters, exception handlers) in isolation. Use MockMvcRequestBuilders to build requests and MockMvcResultMatchers to assert responses.

```
mockMvc.perform(post("/api/products") .contentType(MediaType.APPLICATION_JSON) .content(objectMapper.writeValueAsString(productDto))) .andExpect(status().isCreated()) .andExpect(jsonPath("$.id").exists()) .andExpect(jsonPath("$.name").value("Laptop")) .andDo(print());
```

### Q63. What is the difference between @Mock and @MockBean?

@Mock (Mockito) — creates a pure Mockito mock; no Spring context involved. Used in plain unit tests with @ExtendWith(MockitoExtension.class). @MockBean (Spring Boot Test) — creates a Mockito mock AND registers it as a Spring bean in the ApplicationContext, replacing any existing bean of that type. Used in slice tests (@WebMvcTest) to mock service layer.

```
// Unit test - @Mock @ExtendWith(MockitoExtension.class) class OrderServiceTest { @Mock PaymentRepository paymentRepo; @InjectMocks OrderService orderService; } // Integration/Slice test - @MockBean @WebMvcTest(OrderController.class) class OrderControllerTest { @MockBean OrderService orderService; // Replaces real OrderService in context }
```

### Q64. What is @DataJpaTest? What is its default configuration?

@DataJpaTest loads only JPA components: repositories, entity classes, and JPA configuration. By default, it: uses an in-memory H2 database (replaces configured datasource), enables SQL logging, wraps each test in a transaction that is rolled back after the test (no data leakage between tests). Use @AutoConfigureTestDatabase(replace=NONE) to test against a real database.

```
@DataJpaTest class UserRepositoryTest { @Autowired UserRepository userRepository;
@Autowired TestEntityManager entityManager; @Test void shouldFindByEmail() {
entityManager.persist(new User("Alice", "alice@test.com")); entityManager.flush();
Optional user = userRepository.findByEmail("alice@test.com");
assertThat(user).isPresent(); } }
```

### Q65. What is Testcontainers and how do you use it in Spring Boot?

Testcontainers is a Java library that provides throwaway Docker containers for integration testing. Instead of using H2 (which may behave differently from production DB), you can test against a real MySQL/PostgreSQL instance in Docker. Spring Boot 3.1+ has first-class Testcontainers support with @ServiceConnection.

```
@SpringBootTest @Testcontainers class IntegrationTest { @Container @ServiceConnection
static PostgreSQLContainer postgres = new PostgreSQLContainer<>("postgres:15"); //
Spring Boot auto-configures DataSource from container @Autowired UserRepository
userRepository; @Test void testWithRealPostgres() { /* tests run against real Postgres
*/ } }
```



### Q66. How do you test Spring Boot REST APIs with RestAssured?

RestAssured provides a fluent API for HTTP testing. In Spring Boot, use @SpringBootTest(webEnvironment=RANDOM_PORT) to start a real server on a random port.

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT) class
UserApiTest { @LocalServerPort int port; @BeforeEach void setup() { RestAssured.port =
port; } @Test void shouldCreateUser() { given() .contentType(ContentType.JSON)
.body("{\"name\":\"Alice\",\"email\":\"alice@test.com\"}") .when() .post("/api/users")
.then() .statusCode(201) .body("name", equalTo("Alice")); } }
```

### Q67. What is the use of @TestConfiguration and how is it different from @Configuration?

@TestConfiguration defines additional beans or overrides existing beans specifically for tests. Unlike @Configuration classes (which are auto-detected), @TestConfiguration is NOT automatically picked up by component scanning — it must be explicitly imported using @Import or be a nested class inside the test class. Use it to provide test-specific beans (mocks, stubs, test doubles) without polluting production configuration.

```
@TestConfiguration public class TestConfig { @Bean @Primary public EmailService
mockEmailService() { return new NoOpEmailService(); } } @SpringBootTest
@Import(TestConfig.class) class OrderServiceTest { ... }
```

### Q68. What is @Spy in Mockito? When would you use it over @Mock?

@Mock creates a completely fake object — all methods return default values (null, 0, false) unless stubbed. @Spy wraps a real object — real methods are called by default, but specific methods can be stubbed. Use @Spy when you want to test a real implementation but need to stub only a few methods (e.g., override external service calls while testing real business logic).

```
@Spy OrderService orderService = new OrderService(); @Test void test() {
doReturn(true).when(orderService).sendConfirmationEmail(any()); // Real createOrder()
logic executes; only email is stubbed Order order = orderService.createOrder(dto);
assertThat(order.getStatus()).isEqualTo("CREATED"); }
```

## Section 8: Microservices & Spring Cloud (Q69–Q80)

### Q69. What is a Microservices architecture? What are its advantages and challenges?

Microservices architecture decomposes a monolithic application into small, independent services that: each own their data, are independently deployable, communicate via APIs (REST/messaging). Advantages: independent scaling, technology flexibility, faster deployments, fault isolation, team autonomy. Challenges: distributed system complexity, network latency, data consistency, service discovery, distributed tracing, operational overhead.

### Q70. What is Spring Cloud? Name key Spring Cloud components.

Spring Cloud provides tools for building distributed systems and microservices: Spring Cloud Netflix Eureka — service discovery/registry. Spring Cloud Gateway / Zuul — API gateway and routing. Spring Cloud Config — centralized configuration management. Spring Cloud OpenFeign — declarative REST client. Spring Cloud Sleuth + Zipkin — distributed tracing. Spring Cloud Circuit Breaker (Resilience4j) — fault tolerance. Spring Cloud Bus — event-based config refresh.

### Q71. What is Service Discovery? How does Eureka work?



Service Discovery allows microservices to find each other dynamically without hardcoded URLs. Eureka Server — a registry where services register themselves. Eureka Client — each service registers with Eureka on startup (heartbeat every 30s) and queries Eureka to find other services. Client-side load balancing: Spring Cloud LoadBalancer picks an instance from the registered list.

```
// Eureka Server @SpringBootApplication @EnableEurekaServer public class
DiscoveryServer { } // Eureka Client (other services) # application.properties
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

### Q72. What is Spring Cloud OpenFeign? How does it simplify inter-service communication?

OpenFeign is a declarative HTTP client. Instead of writing RestTemplate boilerplate, you define an interface with annotations, and Spring creates the implementation. It integrates with Eureka (load balancing by service name), Circuit Breaker, and request/response logging.

```
@FeignClient(name = "product-service", fallback = ProductFallback.class) public
interface ProductClient { @GetMapping("/api/products/{id}") Product
getProduct(@PathVariable Long id); @GetMapping("/api/products") List
getAllProducts(@RequestParam int page, @RequestParam int size); } // Usage in another
service @Autowired ProductClient productClient; Product p =
productClient.getProduct(1L); // Eureka resolves 'product-service'
```

### Q73. What is the Circuit Breaker pattern? How do you implement it with Resilience4j?

The Circuit Breaker pattern prevents cascade failures in microservices by stopping calls to a failing service. States: CLOSED (normal, all calls pass), OPEN (calls fail fast without hitting the service), HALF_OPEN (allows test calls to check recovery). Resilience4j provides @CircuitBreaker, @Retry, @RateLimiter, @Bulkhead, @TimeLimiter annotations.

```
@Service public class OrderService { @CircuitBreaker(name = "paymentService",
fallbackMethod = "paymentFallback") @Retry(name = "paymentService") public
PaymentResponse processPayment(PaymentRequest req) { return
```

```
paymentClient.process(req); } private PaymentResponse paymentFallback(PaymentRequest req, Exception ex) { return new PaymentResponse("PENDING", "Payment service unavailable"); } }
```

### Q74. What is an API Gateway? What does Spring Cloud Gateway provide?

An API Gateway is the single entry point for all client requests. It handles routing, authentication, rate limiting, SSL termination, and load balancing. Spring Cloud Gateway: predicate-based routing, global and route-level filters, rate limiting (Redis-based), circuit breaker integration, path rewriting, JWT authentication filter.

```
# application.yml spring: cloud: gateway: routes: - id: user-service uri: lb://user-service predicates: - Path=/api/users/** filters: - StripPrefix=0 - name: CircuitBreaker args: {name: userServiceCB, fallbackUri: forward:/fallback}
```

### Q75. What is the Saga pattern in microservices?



The Saga pattern manages distributed transactions across multiple microservices without a central coordinator using ACID transactions. Two types: Choreography-based Saga — each service publishes events and reacts to others' events. Decentralized, but harder to track. Orchestration-based Saga — a central orchestrator (saga orchestrator) tells each service what to do. Easier to track. Each step has a compensating transaction to undo it if a later step fails (eventual consistency).

### Q76. What is distributed tracing? How do you implement it in Spring Boot?

Distributed tracing tracks a request as it flows through multiple microservices, assigning a unique Trace ID to each request and a Span ID to each service hop. Spring Boot 3.x uses Micrometer Tracing (replaces Spring Cloud Sleuth). Add micrometer-tracing-bridge-otel and zipkin reporter dependencies. Traces are exported to Zipkin/Jaeger for visualization.

```
# application.properties management.tracing.sampling.probability=1.0 # Sample 100% management.zipkin.tracing.endpoint=http://zipkin:9411/api/v2/spans spring.application.name=order-service # Logs automatically include traceId and spanId
```

### Q77. What is the difference between synchronous and asynchronous communication in microservices?

Synchronous — caller waits for response (REST/gRPC). Simple but creates tight coupling and cascade failures. Asynchronous — caller sends message and continues; response comes later (RabbitMQ, Apache Kafka). Decouples services, improves resilience. Use async for: event-driven workflows, high-throughput scenarios, operations that don't need immediate response. Spring Boot integrates with Kafka via spring-kafka and RabbitMQ via spring-boot-starter-amqp.

### Q78. How does Spring Boot integrate with Apache Kafka?

Add spring-kafka dependency. Configure bootstrap servers in application.properties. Use @KafkaListener to consume messages and KafkaTemplate to produce messages.

```
// Producer @Service public class OrderProducer { @Autowired KafkaTemplate kafkaTemplate; public void publishOrder(Order order) { kafkaTemplate.send("order-events", order.getId().toString(), new OrderEvent(order)); } } // Consumer @Service public class NotificationConsumer { @KafkaListener(topics = "order-events", groupId = "notification-group") public void handleOrder(OrderEvent event) { emailService.sendConfirmation(event); } }
```

### Q79. What is Spring Cloud Config? How does it work?

Spring Cloud Config provides centralized external configuration for microservices. Config Server reads from a Git repo (or filesystem) and exposes configuration via REST. Config Clients fetch their config on startup from the Config Server based on {application-name}/{profile}. Supports config refresh without restarting via Spring Cloud Bus + /actuator/refresh.

```
# Config Server @SpringBootApplication @EnableConfigServer public class ConfigServer { } spring.cloud.config.server.git.uri=https://github.com/my/config-repo # Config Client (other services) spring.config.import=configserver:http://config-server:8888 spring.application.name=order-service
```

### Q80. What is the difference between @LoadBalanced RestTemplate and WebClient?

@LoadBalanced RestTemplate — uses Spring Cloud LoadBalancer to resolve service names (e.g., http://user-service/api/users) from Eureka. Blocking/synchronous. WebClient — Spring's reactive, non-blocking HTTP client. Can also be @LoadBalanced. Preferred for reactive/async flows. RestTemplate is deprecated in favor of WebClient in modern Spring Boot applications.

```
// Blocking @Bean @LoadBalanced public RestTemplate restTemplate() { return new RestTemplate(); } // Reactive @Bean @LoadBalanced public WebClient.Builder webClientBuilder() { return WebClient.builder(); } // Usage webClientBuilder.build() .get().uri("http://user-service/api/users/1") .retrieve().bodyToMono(User.class);
```

**@ProminentAcademy**

## Section 9: Performance, Caching & Async (Q81–Q87)

### Q81. How do you implement caching in Spring Boot?

Spring Boot provides declarative caching abstraction with @EnableCaching. Add a cache provider (EhCache, Redis, Caffeine). @Cacheable — caches method result on first call; returns cached value on subsequent calls. @CacheEvict — removes entry from cache. @CachePut — updates cache without skipping method execution.

```
@Service @EnableCaching public class ProductService { @Cacheable(value="products", key="#id", unless="#result == null") public Product getProduct(Long id) { return productRepo.findById(id).orElse(null); } @CacheEvict(value="products", key="#id") public void deleteProduct(Long id) { productRepo.deleteById(id); } @CachePut(value="products", key="#product.id") public Product updateProduct(Product product) { return productRepo.save(product); } }
```

### Q82. How do you integrate Redis cache with Spring Boot?

Add spring-boot-starter-data-redis and spring-boot-starter-cache. Configure Redis host/port. Spring Boot auto-configures RedisCacheManager.



```
# application.properties spring.data.redis.host=localhost spring.data.redis.port=6379 spring.cache.type=redis spring.cache.redis.time-to-live=3600000 # 1 hour in ms # Optional: Configure serializer @Bean public RedisCacheConfiguration cacheConfig() { return RedisCacheConfiguration.defaultCacheConfig() .entryTtl(Duration.ofHours(1)) .serializeValuesWith(RedisSerializationContext.SerializationPair .fromSerializer(new GenericJackson2JsonRedisSerializer())); }
```

### Q83. What is @Async in Spring Boot? How do you enable it and use it?

@Async makes a method run in a separate thread asynchronously. Enable with @EnableAsync on a configuration class. Methods must be public and called from outside the class (proxy limitation). Returns void or Future/CompletableFuture.

```
@Configuration @EnableAsync public class AsyncConfig { @Bean public Executor asyncExecutor() { ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor(); exec.setCorePoolSize(5); exec.setMaxPoolSize(20); exec.setQueueCapacity(100); exec.setThreadNamePrefix("AsyncThread-"); exec.initialize(); return exec; } } @Service public class EmailService { @Async public CompletableFuture sendEmail(String to) { // runs in async thread return CompletableFuture.completedFuture(true); } }
```

### Q84. What is @Scheduled in Spring Boot? How do you configure scheduled tasks?

@Scheduled enables scheduling methods to run at fixed intervals or cron expressions. Enable with @EnableScheduling. fixedRate — runs every N ms, regardless of previous execution. fixedDelay — waits N ms after previous execution completes. cron — uses cron expression for complex scheduling.

```
@Configuration @EnableScheduling public class SchedulingConfig {} @Component public class ReportScheduler { @Scheduled(cron = "0 0 9 * * MON-FRI") // 9 AM weekdays public void generateDailyReport() { ... } @Scheduled(fixedDelay = 60000) // 60s after last execution public void syncData() { ... } @Scheduled(fixedRate = 30000) // Every 30s public void healthCheck() { ... } }
```

### Q85. How do you improve Spring Boot application performance? List key strategies.

1. Use @Async for I/O bound tasks. 2. Enable caching (@Cacheable with Redis/Caffeine). 3. Use LAZY loading in JPA; avoid N+1 queries. 4. Use pagination instead of fetching all records. 5. Use connection pooling (HikariCP — default in Spring Boot). 6. Enable GZip compression (server.compression.enabled=true). 7. Use Spring WebFlux (reactive) for high-concurrency I/O. 8. Use indexed DB columns and optimize JPQL/SQL queries. 9. Minimize ApplicationContext startup time with lazy initialization (spring.main.lazy-initialization=true). 10. Profile with Actuator metrics + Java Flight Recorder.

### Q86. What is HikariCP and how do you configure it in Spring Boot?

HikariCP is the default connection pool in Spring Boot (since 2.0). It is the fastest and most reliable JDBC connection pool. Key configuration properties control pool size, timeouts, and connection validation.

```
# application.properties spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5 spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.pool-name=MyHikariPool
```

***Tip: Pool size formula: (core_count * 2) + effective_spindle_count. For most apps, 10-20 is sufficient.***

### Q87. What is Spring WebFlux? When would you use it over Spring MVC?

Spring WebFlux is the reactive, non-blocking web framework built on Project Reactor. Uses Mono (0-1 element) and Flux (0-N elements) as reactive types. Runs on Netty (non-blocking) instead of Tomcat. Use WebFlux for: high concurrency with I/O bound operations, streaming data, microservices with many external calls. Use Spring MVC for: traditional CRUD apps, blocking I/O (JDBC), simpler development model.

```
@RestController public class ProductController { @GetMapping("/products") public Flux
getAllProducts() { return productService.findAll(); // Returns Flux - non-blocking
stream } @GetMapping("/products/{id}") public Mono getProduct(@PathVariable Long id) {
return productService.findById(id); // Returns Mono - non-blocking single value } }
```

## ■Section 10: Advanced & Scenario-Based Questions (Q88–Q93+)

### Q88. How would you design a rate limiter for a REST API in Spring Boot?

Options: (1) Spring Cloud Gateway with Redis Rate Limiter — built-in, distributed. (2) Bucket4j library — token bucket algorithm, can use Redis for distributed rate limiting. (3) Custom filter using a counter per user IP/ID stored in Redis. Key considerations: per-user vs global limits, sliding window vs fixed window, distributed state for multi-instance deployments.

```
# Spring Cloud Gateway Redis Rate Limiter #
spring.cloud.gateway.routes[0].filters[0].name=RequestRateLimiter #
redis-rate-limiter.replenishRate=10 # req/sec # redis-rate-limiter.burstCapacity=20
```

### Q89. How do you handle transactions across multiple microservices?

Distributed transactions are complex — avoid 2-phase commit (XA) due to performance overhead. Preferred patterns: 1. Saga Pattern — sequence of local transactions with compensating transactions. 2. Outbox Pattern — write event to an 'outbox' table in the same DB transaction, then relay to message broker. 3. Event-Driven eventual consistency — accept temporary inconsistency, resolve asynchronously. Spring Modulith and Spring Integration can help orchestrate these patterns.

### Q90. How would you implement idempotency in a REST API?



Idempotency ensures duplicate requests produce the same result. Critical for payment and order APIs. Implementation: client sends a unique Idempotency-Key header. Server stores (key → response) in Redis. On duplicate request, return cached response without re-processing.

```
@PostMapping("/payments") public ResponseEntity processPayment(
@RequestHeader("Idempotency-Key") String key, @RequestBody PaymentRequest req) { //
Check Redis cache PaymentResponse cached =
redisTemplate.opsForValue().get("idempotency:" + key); if (cached != null) return
ResponseEntity.ok(cached); PaymentResponse response = paymentService.process(req);
redisTemplate.opsForValue().set("idempotency:" + key, response, 24, TimeUnit.HOURS);
return ResponseEntity.status(201).body(response); }
```

### Q91. What are the key differences between Spring Boot 2.x and Spring Boot 3.x?

Spring Boot 3.x (Spring Framework 6): 1. Requires Java 17 minimum (LTS baseline). 2. Jakarta EE 9+ (javax.* → jakarta.* package rename — major migration impact). 3. Native image support via Spring AOT and GraalVM. 4. Micrometer Tracing replaces Spring Cloud Sleuth. 5. @EnableWebSecurity no longer extends WebSecurityConfigurerAdapter (bean-based security config). 6. auto-configuration imports file changed from spring.factories to AutoConfiguration.imports. 7. Spring Data — improved pagination API (PageRequest, ScrollPosition).

■■ **Note:** The javax.* → jakarta.* change is the biggest migration pain point. Update all imports and third-party library versions.

### Q92. How do you containerize a Spring Boot application with Docker?

Option 1: Write a Dockerfile. Option 2: Spring Boot Maven/Gradle plugin with buildpacks (no Dockerfile needed). Best practice: use multi-stage Dockerfile to minimize image size. Use layered JAR (default in Spring Boot 2.3+).

```
# Dockerfile (multi-stage) FROM eclipse-temurin:17-jdk-alpine AS build WORKDIR /app
COPY . . RUN ./mvnw clean package -DskipTests FROM eclipse-temurin:17-jre-alpine
WORKDIR /app COPY --from=build /app/target/*.jar app.jar EXPOSE 8080 ENTRYPOINT
["java","-jar","app.jar"] # OR: Spring Boot Buildpack (no Dockerfile) # mvn
spring-boot:build-image
```

### Q93. What is GraalVM Native Image support in Spring Boot 3? What are the limitations?

Spring Boot 3 supports compiling to a GraalVM native image using Spring AOT (Ahead-of-Time) processing. Native images start in milliseconds (vs seconds for JVM) and use less memory — ideal for serverless/cloud functions. AOT processing generates reflection metadata, proxy classes, and resource hints at build time. Limitations: longer build times, no dynamic class loading at runtime, limited reflection (must be declared), not all libraries are compatible, debugging is harder.

```
# Build native image mvn -Pnative native:compile # Or with Spring Boot plugin mvn
spring-boot:build-image -Pnative # Hint for reflection
@RegisterReflectionForBinding(UserDto.class)
```

## ■ Interview Pro Tips for Spring Boot (3–5 YOE)

1. Always relate your answers to real production scenarios you've worked on.
2. Mention trade-offs — no technology choice is perfect. Interviewers love balanced thinking.
3. For design questions, think out loud: scalability, fault tolerance, consistency.
4. Know at least one full request lifecycle from browser to DB and back.
5. Be ready to write code — MockMvc tests, JPA entities, Security config are common.
6. Stay updated: Spring Boot 3.x, GraalVM, Testcontainers are hot topics in 2024-25.